Развитие цифровых компетенций в рамках дисциплины «Цифровая аналитика»

ПОЛИТЕХ
Санкт-Петербургский
политехнический университет
Петра Великого

Цифровая аналитика

Общепрофессиональный уровень «Цифровая аналитика» учитывает потребности в освоении цифровых компетенций конкретного направления подготовки.
Компетенции данного уровня, в основном, определяют способность личности работать с большими массивами структурированной и неструктурированной информации и включают понятия искусственный интеллект, машинное обучение, нейронные сети, базы данных, основы языков анализа данных.

Лабораторные работы

1. Управление ФС посредством CLI-приложения на Python
2. Извлечение и обработка данных из стороннего web-сервера с помощью Python-приложения
3. Использование Python-библиотек для работы с искусственным интеллектом
4. Взаимодействие с собственным сервисом посредством Telegram-бота

Классификация языков программирования (1)

Процедурные

Низкого уровня

- Ассемблер

Высокого уровня

- Pascal
- C
- Basic
- Fortran

Непроцедурные

Объектно-ориентированные

- C++
- Visual Basic
- Delphi
- Java

Декларативные

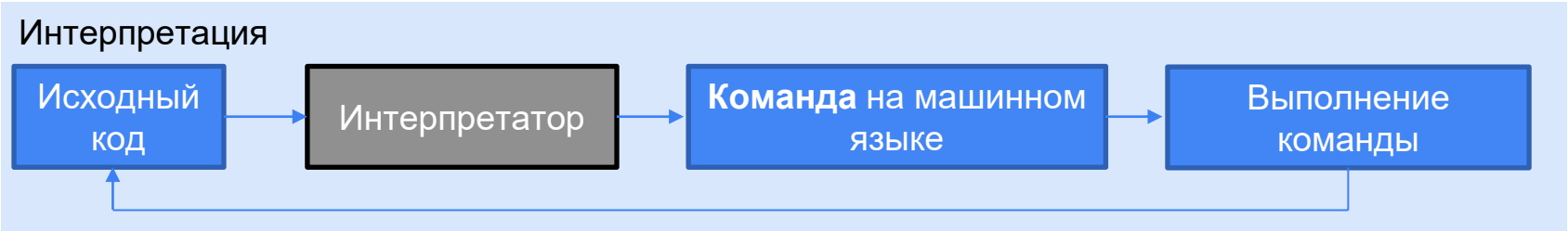
Логические
- Prolog

Функциональные
- Lisp
- F#

Классификация языков программирования (2)

Транслятор – программное средство, преобразующее программный код с языка высокого уровня в машинный код

- Компилятор – Переводит весь исходный код в машинный язык. Генерирует программу
- Интерпретатор – Переводит исходный код «построчно». Напрямую взаимодействует с ОС



Компиляторы и интерпретаторы

Компилируются в машинный код	Компилируются в байт-код	Компилируются в байт-код неявно	Интерпретируются
C, C++, Fortran	**Java, C#**	**Python**	**Perl, Bash**

Компилируемые ⟷ Интерпретируемые

Достоинства и недостатки

Компилятор		Интерпретатор	
Скорость работы Нет необходимости в компиляторе	Переносимость Сложность отладки Время разработки	Переносимость Простота в отладке и диагностике ошибок Размер машинного кода	Не может выполняться без интерпретатора Скорость Потребление памяти

Язык программирования Python

Интерпретируемый

Поддерживает несколько парадигм:
- императивное
- процедурное
- структурное
- объектно-ориентированное
- функциональное

Типизация: динамическая, строгая

Командная строка - python

```
C:\Users\maib-user>python
Python 3.11.0 (main, Oct 24 2022, 18:26:48) [MSC v.1933 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello, World!")
Hello, World!
>>>
```

Низкая скорость работы и более высокое потребление памяти написанных на нём программ

Трансляция кода Python

Виртуальная машина Python (PVM)

Исходный код source.py → Байт-код на python source.pyc → Выполнение (PVM)

Принцип работы:
1. CPython компилирует исходный код в байт-код
2. Байт-код запускается (интерпретируется) на виртуальной машине

Этапы:
- Инициализация - настройку различных структур данных, необходимых python-процессу
- Компиляция - парсинг исходного кода для построения синтаксических деревьев, создание абстрактных синтаксических деревьев, создание таблицы символов и генерацию объектов кода
- Интерпретация - этап фактического выполнения сгенерированных объектов кода в некотором окружении

Аналоги Cpython – Pypy, Jython, IronPython

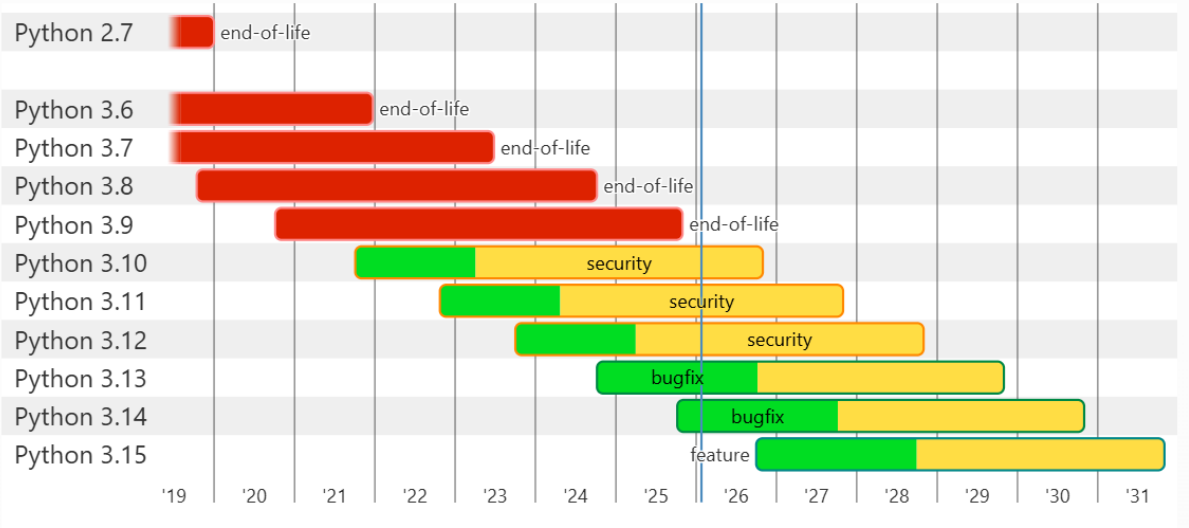
Код:

```
>>> print("Hello, World!")
Hello, World!
```

Байт-код:

```
>>> import dis #импортируем модуль "dis" - Disassembler of Python
>>> dis.dis('print("Hello, World!")')
  1           0 LOAD_NAME                0 (print)
              2 LOAD_CONST               0 ('Hello, World!')
              4 CALL_FUNCTION            1
              6 RETURN_VALUE
```

Версии Python



Командная строка

```
C:\Users\maib-user>python --version
Python 3.11.0

C:\Users\maib-user>
```

Режимы работы интерпретатора Python

Интерактивный режим

Командная строка

```
C:\Users\maib-user>python
Python 3.11.0 (main, Oct 24 2022, 18:26:48) [MSC
Type "help", "copyright", "credits" or "license"
>>> import base64
>>> s = "Hello World!"
>>> b = s.encode("UTF-8")
>>> e = base64.b64encode(b)
>>> s1 = e.decode("UTF-8")
>>> print(s1)
SGVsbG8gV29ybGQh
>>> quit()
```

```
C:\Users\maib-user>python
Python 3.11.0 (main, Oct 24 2022,
Type "help", "copyright", "credits
>>> a=5
>>> b=6
>>> print(a+b)
11
>>> a=1
>>> print(a+b)
7
>>>
```

Программный режим

File Edit Selection View Go Run ···

test.py × auto_script.py 8

C: > Google Drive > prep > цифровая культура > test.py > ...

```
1  import base64
2
3  if __name__ == "__main__":
4      s = "Hello World!"
5      b = s.encode("UTF-8")
6      e = base64.b64encode(b)
7      s1 = e.decode("UTF-8")
8      print(s1)
```

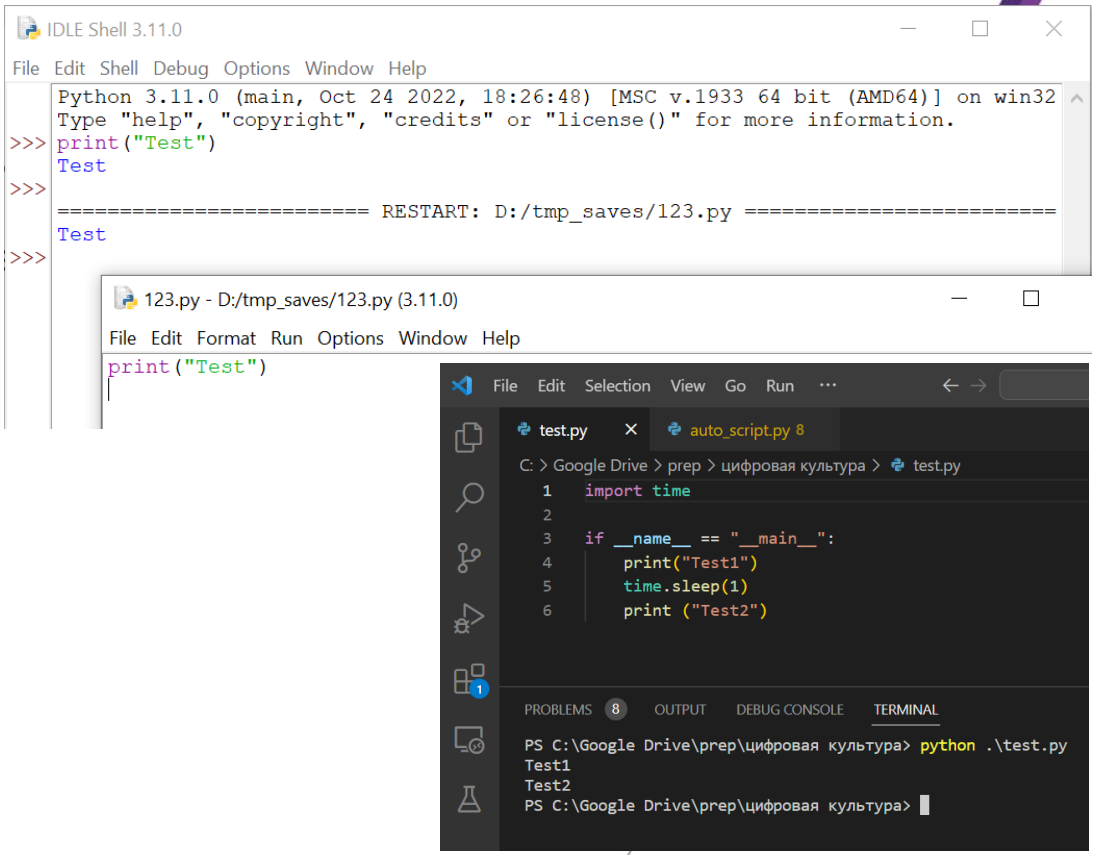
PROBLEMS 8 OUTPUT DEBUG CONSOLE TERMINAL

```
PS C:\Google Drive\prep\цифровая культура> python .\test.py
SGVsbG8gV29ybGQh
PS C:\Google Drive\prep\цифровая культура>
```

Среды разработки Python

IDE – Интегрированная среда разработки (Integrated Development Enviroment):

- IDLE
- PyDev
- Aptana
- **PyCharm**
- Eclipse
- Emacs
- Vim
- NotePad++
- **Visual Studio Code**



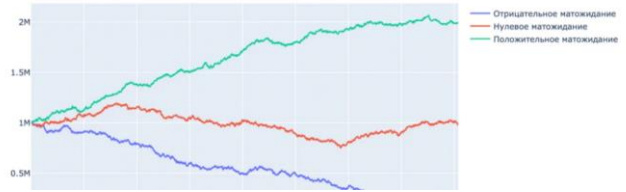
Jupyter Notebook

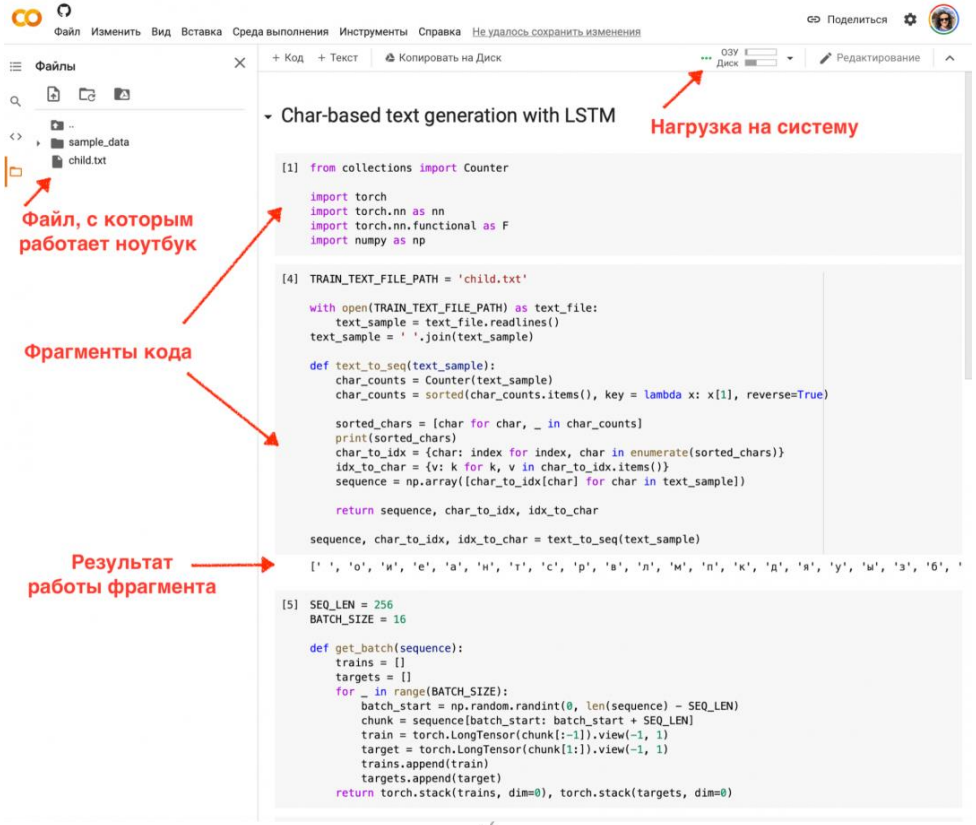
Среда разработки с выводом результатов в процессе выполнения кода

- используется в Data Science для визуализации данных
- есть возможность запуска локально так и в облаке
- поддержка Ruby, Perl, R

```
fig.add_trace(go.Scatter(x=games2, y=balance2, name = "Нулевое матожидание"))
# и для третьей
fig.add_trace(go.Scatter(x=games3, y=balance3, name = "Положительное матожидание"))
# выводим графики в браузер
fig.show()
```

```
Выиграно ставок: 19696 (48.76213111507229%).  Проиграно ставок: 20696 (51.23786888492771%).
Выиграно ставок: 20187 (49.9777183600713%).  Проиграно ставок: 20205 (50.0222816399287%).
Выиграно ставок: 20695 (51.23539314715785%).  Проиграно ставок: 19697 (48.76460685284215%).
```





Установка модулей (пакетов)

1. Установка из исходников: python setup.py install
2. Использование программы pip (python get-pip.py)
   #pip install package_name – установка пакета
   #pip install -U <PACKAGE> – обновление пакета
   #pip list – вывести список установленных пакетов и их версий
   #pip uninstall – удаление пакета
   #pip install <PACKAGE>==<VERSION> - установка определенной версии

Командная строка - python

```
C:\Users\maib-user>python
Python 3.11.0 (main, Oct 24 2022, 18:26:48) [M
Type "help", "copyright", "credits" or "licens
>>> import scapy
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'scapy'
>>>
```

```
C:\Users\maib-user>pip install scapy
Collecting scapy
  Downloading scapy-2.5.0.tar.gz (1.3 MB)
     ---------------------------------------- 1.3/1.3 MB 4.3 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Installing collected packages: scapy
  DEPRECATION: scapy is being installed using the legacy 'setup.py install' met
t.toml' and the 'wheel' package is not installed. pip 23.1 will enforce this be
 to enable the '--use-pep517' option. Discussion can be found at https://github
  Running setup.py install for scapy ... done
Successfully installed scapy-2.5.0
```

Виртуальное окружение (virtual environment)

Механизм изоляции зависимостей проекта от глобального окружения
Предотвращает возможные конфликты между библиотеками

- venv
- pyenv-virtualenv
- virtualenv
- Poetry
- Pipenv
- Pyflow
- Conda
- Hatch

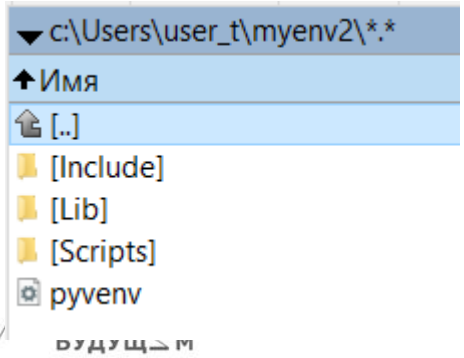
- *python -m venv myenv2* – создание виртуального окружения myenv2
- *myenv2\Scripts\activate* – активация окружения
- *deactivate* – выход из окружения

Глобальное окружение

```
C:\Users\user_t>pip list
Package                      Version
---------------------------- --------
absl-py                      2.1.0
anyio                        3.7.0
argon2-cffi                  21.3.0
argon2-cffi-bindings         21.2.0
arrow                        1.2.3
asttokens                    2.2.1
astunparse                   1.6.3
async-lru                    2.0.2
attrs                        23.1.0
Babel                        2.12.1
backcall                     0.2.0
```

Виртуальное окружение

```
C:\Users\user_t>python -m venv myenv2

C:\Users\user_t>myenv2\Scripts\activate

(myenv2) C:\Users\user_t>pip list
Package    Version
---------- -------
pip        22.3.1
setuptools 65.5.0
```



Примеры работы с Python

C: > tmp_saves > test.py > ...

```python

if __name__=="__main__":
    var_1 = 15
    var_2 = 67
    print('var_1:', var_1, 'var_2:', var_2)
    var_temp = var_1
    var_1 = var_2
    var_2 = var_temp
    print('var_1:', var_1, 'var_2:', var_2)
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

```
PS C:\tmp_saves> python .\test.py
var_1: 15 var_2: 67
var_1: 67 var_2: 15
PS C:\tmp_saves>
```

C: > tmp_saves > test.py > ...

```python

if __name__=="__main__":
    var_1 = 15
    var_2 = 67
    print('var_1:', var_1, 'var_2:', var_2)
    var_1, var_2 = var_2, var_1
    print('var_1:', var_1, 'var_2:', var_2)
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

```
PS C:\tmp_saves> python .\test.py
var_1: 15 var_2: 67
var_1: 67 var_2: 15
```

C: > tmp_saves > test.py > ...

```python

if __name__=="__main__":
    var_1 = [1, 5, 6, 7]
    sum = 0
    for i in var_1:
        sum += i
    print(sum/len(var_1))
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

```
PS C:\tmp_saves> python .\test.py
4.75
```

C: > tmp_saves > test.py > ...

```python

if __name__=="__main__":
    var_1 = [1, 5, 6, 7]
    print(eval(str(var_1).replace(', ', '+'))[0]/len(var_1))
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

```
PS C:\tmp_saves> python .\test.py
4.75
```

C: > tmp_saves > test.py > ...

```python

if __name__=="__main__":
    var_1 = 11
    print(var_1 in [5, 6, 40, 126])
    print(40 in [5, 6, 40, 126])
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

```
PS C:\tmp_saves> python .\test.py
False
True
```

Основные структуры данных

Списки (list) – изменяемые, упорядоченные коллекции

```python
if __name__ == "__main__":
    # Создание списка
    numbers = [1, 2, 3, 4, 5]
    print (type(numbers))
    print(numbers)

    # Добавление элемента
    numbers.append(6)

    # Удаление элемента
    numbers.remove(3)

    # Доступ по индексу
    print(numbers[2])

    # Перебор списка
    for num in numbers:
        print(num)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS C:\Google Drive\prep\цифровая культура> python .\test.py
<class 'list'>
[1, 2, 3, 4, 5]
4
1
2
4
5
6
```

Словари (dict) – хранение данных в формате "ключ-значение"

```python
if __name__ == "__main__":
    # Создание словаря
    student = {
        "name": "Иван",
        "age": 22,
        "course": "Python"
    }

    # Доступ к значению
    print(student["name"])  # Анна

    # Добавление нового ключа
    student["city"] = "Санкт-Петербург"

    # Перебор ключей и значений
    for key, value in student.items():
        print(f"{key}: {value}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS C:\Google Drive\prep\цифровая культура> python .\test.py
Иван
name: Иван
age: 22
course: Python
city: Санкт-Петербург
```

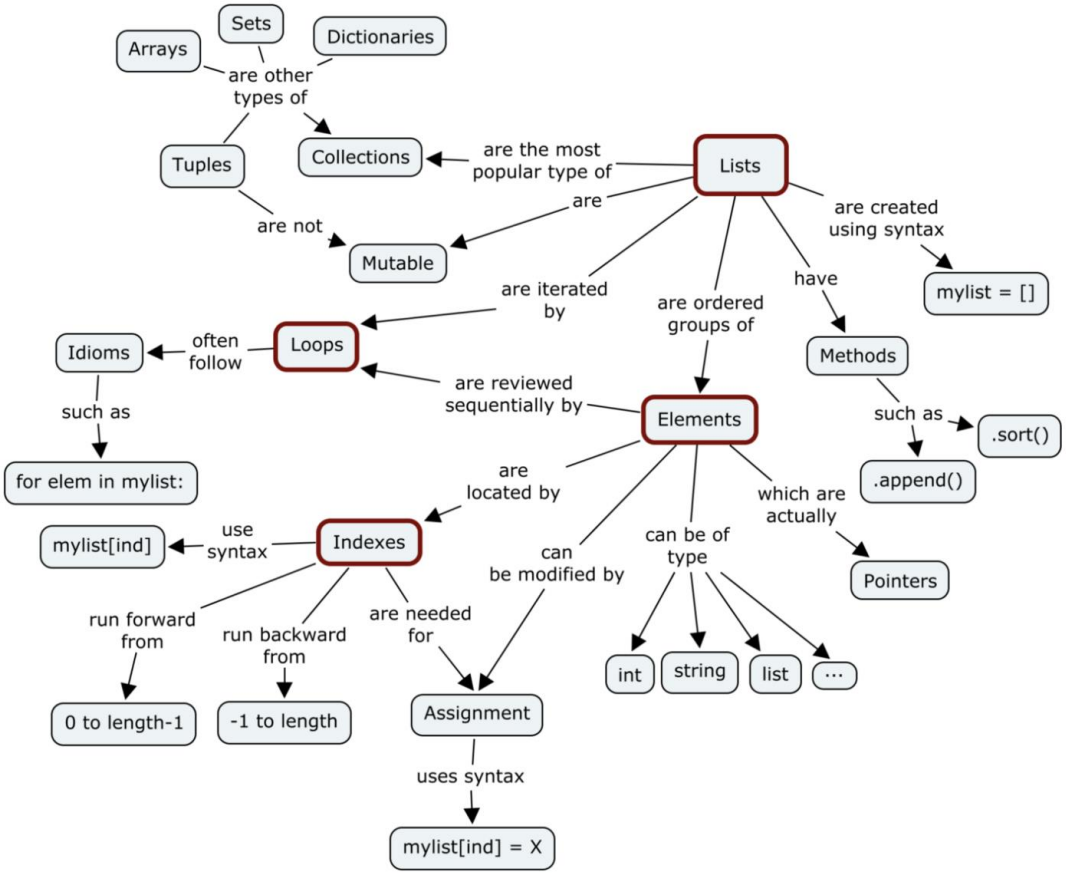
Множества (set) – коллекция уникальных элементов

```python
import base64

if __name__ == "__main__":
    # Создание множества
    unique_numbers = {1, 2, 3, 3, 4, 5}

    # Добавление элемента
    unique_numbers.add(6)

    # Удаление элемента
    unique_numbers.remove(2)

    print(unique_numbers)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS C:\Google Drive\prep\цифровая культура> python .\test.py
{1, 3, 4, 5, 6}
```

Списки (list) в Python: структура, свойства и операции



Основные конструкции

Объявление и вызов функции

```python
def greet(name):
    return f"Привет, {name}!"

print(greet("Иван"))


def add_numbers(*args):
    return sum(args)

print(add_numbers(1, 2, 3, 4))  # 10
```

ООП

```python
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def info(self):
        return f"{self.brand} {self.model}"

car = Car("Toyota", "Camry")
print(car.info())
```

Работа с файлами

```python
with open("example.txt", "r") as file:
    content = file.read()
    print(content)

with open("example.txt", "w") as file:
    file.write("Привет, мир!")
```

Обработка исключений

```python
try:
    num = int(input("Введите число: "))
    print(10 / num)
except ZeroDivisionError:
    print("Ошибка: деление на ноль!")
except ValueError:
    print("Ошибка: введите число!")
finally:
    print("Завершение программы")
```

МЫСЛ ТЬ
УДУЩ М

PEP8

PEP – Python Enhancement Proposal («Предложение по улучшению Python»). Это документы, предлагающие новые особенности языка. PEP8 — это официальное руководство по стилю кода Python.

- используйте 4 пробела для отступа и не используйте табы, не смешивайте их
- максимальная длина строки 79 символов; для разрыва строки используйте отступы или бэкслеш
- функции верхнего уровня и определения классов отделяйте двумя пустыми строками
- определения методов внутри класса отделяйте одной пустой строкой
- дополнительные пустые строки используйте для логической группировки методов
- кодировка файлов должна быть Latin-1, а в Питоне 3 и выше — utf-8
- каждый импортируемый модуль с новой строки
- порядок импортов: стандартные модули, third-party модули, локальные модули приложения
- пустая строка между каждой группой импортов
- избегайте лишних пробелов внутри скобок; перед запятыми, точкой с запятой и двоеточиями
- избегайте лишних пробелов перед скобками с аргументами функций и скобками с индексами
- избегайте больше чем одного пробела между операторами (= — + итд)
- используйте одиночный пробел для выделения операторов
- избегайте пробелов вокруг '=', который используется для параметров по-умолчанию и keyword аргументов
- обновляйте комментарии вместе с кодом
- пишите комментарии по-английски
- избегайте имён из одиночных l, O, I

PEP8 (2)

Отступы

```
# Выровнено по открывающему разделителю
foo = long_function_name(var_one, var_two,
                         var_three, var_four)

# Больше отступов включено для отличения его от остальных
def long_function_name(
        var_one, var_two, var_three,
        var_four):
    print(var_one)
```

```
# Аргументы на первой линии запрещены, если не используется вертикальное выравнивание
foo = long_function_name(var_one, var_two,
    var_three, var_four)

# Больше отступов требуется, для отличения его от остальных
def long_function_name(
    var_one, var_two, var_three,
    var_four):
    print(var_one)
```

Пробелы - самый предпочтительный метод отступов.
Табуляция должна использоваться только для поддержки кода, написанного с отступами с помощью табуляции.

Длина строки – 79 символов

```
with open('/path/to/some/file/you/want/to/read') as file_1, \
        open('/path/to/some/file/being/written', 'w') as file_2:
    file_2.write(file_1.read())
```

Отделяйте функции верхнего уровня и определения классов двумя пустыми строками.
Определения методов внутри класса разделяются одной пустой строкой.

PEP8 (3)

Пробелы

```
mix_arr = ("John", [176, 70], {"age" : 14})
mix_arr[0:2]
mix_arr[0 : 2]
custom_function(args)
weight = 70
height = 1,76
body_mass_index = 22,5
```

```
mix_arr = ( "John", [ 176, 70 ], { "age" : 14 } )
mix_arr[0 :2]
mix_arr[0: 2]
custom_function (args)
weight          = 70
height          = 1,76
body_mass_index = 22,5
```

Комментарии

```
# Comment1
a = 5
b = 10
a = b - a + a

a = a / 2 #Comment2

"""Speed determination.

Keyword arguments:
distance -- distance traveled in meters
time -- time in seconds (default 1)
"""
```

The Zen of Python

```
C:\Users\maib-user>python
Python 3.11.6 (tags/v3.11.6:8b6ee5b, Oct  2 2023, 14:57:12) [MSC v.1935 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import this
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```